EXERCICE 04 / 4

LA PERSISTANCE

PostgreSQL · Sequelize ORM · Railway · Connexion React

Difficulte : ★★★★ 24 heures

Contexte pedagogique

Le fichier JSON est suffisant pour apprendre mais insuffisant en production. Cet exercice final connecte l'API a une vraie base de donnees relationnelle PostgreSQL via Sequelize ORM. L'API est deployee sur Railway avec la base de donnees. Et pour la premiere fois, le Dashboard React de la Section 2 consomme de vraies donnees persistantes — la boucle est bouclée.

Regle EX04 — Projet final Section 3

L'apprenant peut librement refactoriser et ameliorer son code des exercices precedents.
L'objectif est une API deployee en production avec une vraie base de donnees.
Le lien entre le React Dashboard et cette API est le livrable le plus important.

Competences ciblees

- PostgreSQL : bases du SQL, types de donnees, relations
- Sequelize ORM : modeles, migrations, associations, requetes
- Connexion a PostgreSQL depuis Node.js avec Sequelize
- Migrer de fichier JSON vers base de donnees (remplacer les controllers)
- Variables d'environnement : DATABASE_URL pour Railway
- Deployer sur Railway : Node.js + PostgreSQL dans le meme projet
- Connecter le Dashboard React (Vercel) a l'API (Railway) — CORS en production
- Tester la chaine complete : React → API → PostgreSQL

CONFIGURATION MOODLE — Parametres de l'activite

```
Type d'activite  : Devoir (Assignment) — PROJET FINAL SECTION 3
Titre            : EX04 — La Persistance : PostgreSQL + Railway + React connecte
Prerequis        : EX03 valide >= 6/10
Delai            : 24 heures apres deblocage
Rendu            : URL Railway API publique + Dashboard React mis a jour sur Vercel
Note max         : 10 points
Badge Moodle     : Badge Node.js Developer — attribue automatiquement a >= 8/10
Ressources       : Video Sequelize (20min) + Guide Railway (PDF) + Video connexion React-Node
```

1 PostgreSQL avec Sequelize ORM

```
// Terminal + JavaScript — Installation et configuration Sequelize
npm install sequelize pg pg-hstore
npm install --save-dev sequelize-cli

// src/config/database.js
const { Sequelize } = require('sequelize');

const sequelize = new Sequelize(process.env.DATABASE_URL, {
  dialect: 'postgres',
  logging: process.env.NODE_ENV === 'development' ? console.log : false,
  dialectOptions: {
    ssl: process.env.NODE_ENV === 'production'
      ? { require: true, rejectUnauthorized: false }
      : false,
  },
});

module.exports = sequelize;

// src/models/Sensor.js
const { DataTypes } = require('sequelize');
const sequelize      = require('../config/database');

const Sensor = sequelize.define('Sensor', {
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  name:      { type: DataTypes.STRING(50), allowNull: false },
  type:      { type:
DataTypes.ENUM('temperature','humidity','light','pressure'), allowNull:
false },
  value:     { type: DataTypes.FLOAT, defaultValue: 0 },
  unit:      { type: DataTypes.STRING(10), defaultValue: '' },
  isActive:  { type: DataTypes.BOOLEAN, defaultValue: true },
  userId:    { type: DataTypes.INTEGER, allowNull: false }, // Cle
etrangere
}, { tableName: 'sensors', timestamps: true });

module.exports = Sensor;
```

- DataTypes.ENUM garantit que seules les valeurs autorisees sont acceptees au niveau de la base de donnees
- timestamps: true ajoute automatiquement createdAt et updatedAt a chaque enregistrement
- La cle etrangere userId lie chaque capteur a son proprietaire — un apprenant ne voit que ses capteurs

2 Modele User et associations

```
// JavaScript — Modele User et association avec Sensor
// src/models/User.js
const { DataTypes } = require('sequelize');
const sequelize      = require('../config/database');
```

```
const bcrypt         = require('bcryptjs');

const User = sequelize.define('User', {
  id:       { type: DataTypes.INTEGER, autoIncrement: true, primaryKey:
true },
  name:     { type: DataTypes.STRING,  allowNull: false },
  email:    { type: DataTypes.STRING,  allowNull: false, unique: true,
validate: { isEmail: true } },
  password: { type: DataTypes.STRING,  allowNull: false },
}, { tableName: 'users', timestamps: true });

// Hook : hacher le mot de passe avant chaque sauvegarde
User.beforeCreate(async (user) => {
  user.password = await bcrypt.hash(user.password, 12);
});

// Associations
User.hasMany(Sensor,   { foreignKey: 'userId', as: 'sensors' });
Sensor.belongsTo(User, { foreignKey: 'userId', as: 'owner' });

// src/server.js — Synchroniser les modeles avec la base
const sequelize = require('./config/database');
require('./models/User');
require('./models/Sensor');

sequelize.sync({ alter: true }) // alter: true met a jour les tables sans
les supprimer
  .then(() => console.log('Base de donnees synchronisee'))
  .catch(err => console.error('Erreur BDD :', err));
```

- User.beforeCreate : le hook Sequelize hache le mot de passe automatiquement — plus besoin de le faire dans le controller
- hasMany / belongsTo : definit la relation 1-N entre User et Sensor
- sync({ alter: true }) : met a jour la structure des tables sans perdre les donnees

3 Deploiement sur Railway

```
// Terminal + Configuration — Deploiement Railway
// 1. Creer un compte sur railway.app
// 2. Installer la CLI Railway
npm install -g @railway/cli
railway login

// 3. Creer un nouveau projet
railway init

// 4. Ajouter le plugin PostgreSQL dans le dashboard Railway
// Dashboard Railway → New → Database → PostgreSQL
// Railway genere automatiquement DATABASE_URL

// 5. Configurer les variables d'environnement dans Railway
// Variables a ajouter dans le dashboard Railway :
// NODE_ENV=production
```

```
// JWT_SECRET=une_cle_secrete_longue_et_aleatoire
// JWT_EXPIRATION=24h
// FRONTEND_URL=https://votre-portfolio.vercel.app

// 6. Deployer
railway up

// railway.toml — Configuration du build
[build]
builder = 'nixpacks'

[deploy]
startCommand = 'npm start'
healthcheckPath = '/api/health'

// Ajouter une route de healthcheck dans app.js :
app.get('/api/health', (req, res) => {
  res.json({ status: 'OK', timestamp: new Date().toISOString() });
});
```

- Railway est la plateforme la plus simple pour deployer Node.js + PostgreSQL en meme temps
- Ne jamais commiter les variables d'environnement — les configurer dans le dashboard Railway
- La route /api/health permet de verifier que le serveur repond correctement

4 Connexion du Dashboard React a l'API deployee

Derniere etape : modifier le Dashboard React de la Section 2 pour consommer la vraie API au lieu des donnees simulees.

```
// JavaScript — Connexion React vers l'API Railway
// Dans le projet React (Section 2)
npm install axios

// src/services/api.js
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3000/api',
  timeout: 10000,
});

// Intercepteur : ajouter le token JWT a chaque requete
api.interceptors.request.use(config => {
  const token = localStorage.getItem('jwt_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export const sensorsApi = {
  getAll:    ()        => api.get('/sensors'),
  getById:   (id)      => api.get(`/sensors/${id}`),
```

```
  create:    (data)    => api.post('/sensors', data),
  update:    (id, d)   => api.put(`/sensors/${id}`, d),
  remove:    (id)      => api.delete(`/sensors/${id}`),
};

// Dans le fichier .env du projet React :
// VITE_API_URL=https://votre-api.railway.app/api

// Remplacer le useIoTSimulator de EX02 par des vraies donnees :
const [sensors, setSensors] = useState([]);

useEffect(() => {
  sensorsApi.getAll()
    .then(res => setSensors(res.data.data))
    .catch(err => console.error('Erreur API :', err));
}, []);
```

- VITE_API_URL dans .env du projet React — a ajouter aussi dans les variables d'environnement Vercel
- L'intercepteur axios ajoute automatiquement le token a toutes les requetes — plus besoin de le faire manuellement
- Tester la chaine complete : creer un compte via Swagger, s'authentifier, ajouter des capteurs, les voir dans le Dashboard React